

Wingman

A Jetson GPU perceives hazards; a PHYTEC phyBOARD-RT1170 makes the safety decision — deterministically — and fails safe the moment perception goes silent.

ADAS *prototype* (proof of concept, not a certified system) — All About Circuits / PHYTEC / NXP 2026 Embedded Design Contest, built around the PHYTEC phyBOARD-RT1170 (phyCORE-RT1170 SoM, NXP i.MX RT1176).

The Problem

GPU-based perception is fast and powerful, but it is not deterministic: it can stall on a bad frame, crash, drop the camera, or simply take longer than expected under load. A safety system built entirely on that foundation inherits its uncertainty. Real vehicles solve this by splitting the problem: a perception computer figures out *what* is out there, and a separate, simpler, hardware-watchdogged ECU decides *whether it's dangerous* and what to do if the eyes go dark. Wingman is that split, built at prototype scale.

Architecture

	Perception (Jetson TX2) — the eyes	Supervisor (phyBOARD-RT1170) — the reflexes
Runs	Linux + CUDA / TensorRT	Zephyr RTOS on a 996 MHz Cortex-M7
Good at	heavy, fast, parallel vision	bounded-time decisions, instant boot
Failure mode	can stall, crash, drop frames	watched by a hardware watchdog
Owns	<i>what</i> is out there	<i>whether it's dangerous</i> , and what to do if the eyes go dark

```
[0V5693 CSI cam] -> Jetson TX2: detectNet (TensorRT) -> pinhole distance -> Kalman tracker -> UDP
perception frame @30 Hz --Ethernet--> phyBOARD-RT1170 (Zephyr, Cortex-M7) validate (len/CRC/seq) ->
TTC -> state machine -> LEDs + probe GPIOs, 10 ms heartbeat -> FAILSAFE hardware watchdog, onboard
IMU context <----- UDP status (verdict, TTC, latency) ---' The TX2 draws boxes in the
colour of the RT1170's verdict.
```

Why the phyBOARD-RT1170

Wingman deliberately puts the phyBOARD-RT1170 where its strengths matter most: as the deterministic authority a GPU-based perception system reports *to*, not the other way around. Every safety decision, and the fail-safe response when perception goes dark, happens entirely on the RT1170's Cortex-M7 — independent of whether the Linux/GPU side is even alive.

Deterministic, cycle-accurate timing. Decision latency is measured with the M7's own 996 MHz cycle counter (`k_cycle_get_32`), not wall-clock estimates — mean 49.79 μ s, p99 118.62 μ s, max 205.10 μ s across 1,614 real frames.

Hardware watchdog (WDOG1). Fed only by the highest-priority decision thread. Demonstrated live via a deliberate thread stall (`wm hang`): the board resets within ~0.5 s and reports the reset cause, read back from the SoC's own SRC register, over the wire on the very next boot.

Real-time Ethernet link. The board's 1 Gbit ENET_1G interface (confirmed full-duplex, carrier up) carries a custom low-latency UDP protocol to the TX2 — round trip averages 0.87 ms, max 2.02 ms across 1,614 frames.

Fail-safe by design, not by afterthought. Any loss of valid perception data — network flood, a crashed camera pipeline, corrupted frames — trips FAILSAFE within one heartbeat window. Measured over 50 real trials: mean 107.26 ms, max 112.36 ms, landing exactly on the "~100 ms + one 10 ms heartbeat tick" design prediction.

Onboard IMU (ICM-40627) integration. Hit and fixed a real bug in this board's PHYTEC BSP branch (`v4.1.0-phy2`): the IMU's devicetree entry ships at the wrong I2C address. PHYTEC's own upstream repository already carried the fix on a newer branch; we traced it and applied the same one-line address correction via a targeted devicetree overlay rather than a full BSP upgrade this close to the deadline. Axis behavior was then verified against real physical motion.

Full test matrix on real silicon. 14 of 14 scenarios passed on the physical RT1170 — every scripted scenario plus every fault-injection variant (packet drop, CRC corruption, reorder, duplicate, rate change, burst-loss, simulated perception crash) — matching the board's own `native_sim` result exactly.

The Jetson TX2 supplies the perception input and is, deliberately, supporting cast: every number above about *safety* was earned on the phyBOARD-RT1170 alone.

How It Works

Protocol

146-byte perception frames (up to 8 tracks: id, class, confidence, distance, lateral offset, closing speed) and 36-byte status replies, little-endian, CRC-16/CCITT-FALSE, sequence numbers. Frames are sent every camera frame even with zero tracks — they *are* the heartbeat the supervisor watches.

Supervisor threads (RT1170)

`net_rx` stamps each datagram with the cycle counter the instant the socket delivers it, validates length/CRC/magic/version, and queues it. `decide` (highest priority) runs the state machine, writes the LEDs, measures latency, and feeds the watchdog. A 10 ms timer detects heartbeat loss. `status_tx` replies to whoever sent the frame.

Decision logic

In-path tracks ($|\text{lateral}| < 1.5$ m, confidence ≥ 0.4 , closing > 0.1 m/s); time-to-collision = distance $\div$ closing speed; the minimum-TTC track is the threat. Hysteresis: CAUTION below 2.5 s (exit above 2.8 s), WARNING below 1.2 s (exit above 1.5 s), plus a close-range override (< 3 m and closing) for hazards too near for TTC math to matter. **Any state** $\rightarrow$ **FAILSAFE** after 100 ms without a valid frame, or 5 consecutive corrupt frames; recovery only after 10 consecutive in-sequence frames.

IMU context

Sustained braking (< -0.3 g for 200 ms) downgrades WARNING to CAUTION — the driver is already reacting — except during the close-range override, where it never suppresses. A swerve during an active threat is logged as a near-miss; a > 2 g spike freezes a 64-decision black box.

Perception (TX2)

SSD-MobileNet-v2 via TensorRT; monocular pinhole distance ($\text{dist} = f_{\text{px}} \cdot H_{\text{class}} / \text{box_height}$) with focal length measured by `calibrate.py`; constant-velocity Kalman tracker with gated nearest-neighbour association, requiring 3 consecutive hits before a track is trusted.

Results

All numbers below were measured on real hardware — the physical phyBOARD-RT1170 and Jetson TX2 — not simulated. Raw CSVs and the full scenario matrix are in the repository's `docs/` folder.

Metric	Result
Decision latency (socket delivery $\rightarrow$ outputs)	n=1,614 min 11.92 / mean 49.79 / p99 118.62 / max 205.10 μ s
Failover time (last valid frame $\rightarrow$ FAILSAFE)	n=50, 50/50 passed min 102.32 / mean 107.26 / p99 112.36 / max 112.36 ms
Round trip (TX2 $\rightarrow$ RT1170 $\rightarrow$ TX2)	n=1,614 min 0.50 / mean 0.87 / p99 1.37 / max 2.02 ms
Scenario test matrix	14/14 passed — 6 scenarios $\times$ every fault-injection variant, matches native_sim

Latency under network flood (5,000 pps)	RT1170 decision latency stays flat: 51.19 vs 47.46 μ s baseline. Real host-side packet-delivery gap found during flood's active window (21/30 vs 20/20 baseline) — traced to TX2 CPU/socket contention between two Python processes, not an RT1170 issue.
Perception FPS	steady ~30.2 fps average (441 samples) over a 10-minute continuous live-camera soak, zero crashes
CUDA Kalman kernel (Tegra X2, sm_62)	Loses to CPU at N=64 (0.91 $\times$, as expected from launch overhead); crossover at N=256 (3.33 $\times$); scales to 59-61 $\times$ at N $\geq$ 16,384. SoA vs AoS: 100% vs 14.04%/12.50% memory-coalescing efficiency (nvprof), similar occupancy (~0.85 both)



Fig. 1 — A real finding, not a staged shot: at the working distance available, a full standing person's feet fall outside the camera frame. This directly motivated the calibration workaround described under Limitations, and is left in this document as evidence rather than cropped out.

Honest Limitations

- Monocular distance estimation assumes a fixed real-world height per object class; truncated or unusually-sized objects will read wrong.
- The test space could not fit a full standing person in frame at any usable distance (Fig. 1), so the standard 3-distance person-based calibration procedure could not be used. A known-size-object workaround (a sheet of letter paper at two hand-held distances) was used instead, with a documented 13.3% spread between the two measurements — above the tooling's own 10% warning threshold, most likely from hand-held tilt imprecision. Distance/TTC numbers should be read as approximate, not precision-verified.
- Object tracking becomes unstable at extreme close range (roughly arm's length): the detector intermittently loses lock, which can cause the state machine to revert to NOMINAL abruptly — the detection was lost, not the threat resolved. A real characteristic of monocular detection at very close proximity, not a state-machine defect.
- The IMU's sign convention (which physical direction reads as "braking" vs. "accelerating") depends on which edge of the board faces vehicle-front in the final mounting. Left unconfigured deliberately, since it is a mounting decision, not a firmware bug — *axis assignment* (which channel responds to pitch vs. roll) was verified correct by physically moving the board and reading live IMU output.

- Decision-latency numbers above are the RT1170's own self-reported cycle-counter figures; no external logic analyzer was available to independently cross-check them against the two provided GPIO probe points.
- This is a bench prototype evaluated on live and replayed footage. It has not been tested in traffic, is not ISO 26262 qualified, and is not a product.

Future Work

Redo camera calibration with the standard person-based method in a larger space for verified distance/TTC accuracy.

Independent logic-analyzer cross-check of the decision-latency and failover measurements.

Resolve IMU mounting orientation and lock in the sign convention for a specific vehicle install.

TinyML driving-maneuver classifier on IMU windows (steady / brake / accelerate / swerve), scaffolded but not built due to time.

M4 core as an independent hardware monitor of the M7's own heartbeat (dual-core redundancy), using the RT1170's Messaging Unit / IPM.

Radar or GPS fusion, and a custom carrier board sized to the 30×30 mm phyCORE-RT1170 SoM.

Bill of Materials

Category	Items
Hardware	PHYTEC phyBOARD-RT1170 dev kit (+5 V USB-C PSU) • NVIDIA Jetson TX2 developer kit (+19 V PSU) • Cat5e/6 Ethernet cable • 2× micro-USB cables (console + USB-OTG flashing) • OV5693 CSI camera (TX2 kit)
Software / Toolchain	Zephyr 4.1 (PHYTEC BSP v4.1.0-phy2) • Zephyr SDK 0.17.0 • NXP SPSDK / blhost (USB-OTG flashing, no debug probe) • JetPack 4.6.6 (L4T R32.7.6, CUDA 10.2, TensorRT 8.2) • jetson-inference • Python 3

Source and Raw Data

Full source, build/flash instructions, the complete bring-up runbook, and every raw measurement CSV referenced above: github.com/SiliconBeast/wingman